SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	Sri Raghavi N
Register Number	1924260281D
Course Code & Name	CSA02 - C Programming
Slot	_____
Assignment / Task Title	Assignment-3 (Q1): Design, Implement and Validate a Reliable Billing Program for a Rural Electricity Co-operative - Tasks 1 to 4
GitHub Repository URL	https://github.com/<your-username>/CSA02-1924260281D-Assignment...
Submission Date	27-10-2026

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

Environment: gcc (Ubuntu 13.3.0), compiled with `-Wall -Wextra -std=c11`; 64-bit system (sizeof(int)=4, sizeof(long)=8).

TASK 1 - Understand the Problem and Identify the Data (4 Marks, K4)

Task / Problem Statement

A rural electricity co-operative bills households from two meter readings. Analyse the billing problem and identify every item of data the program must store and compute: inputs, outputs, constants, values that can be negative / fractional / very large, the rules every correct bill must satisfy, and the assumptions made. Each item must be justified.

Aim

To analyse the billing problem completely before writing code, so that data types, constants and macros in later tasks are chosen on evidence rather than guesswork.

Concept / Theory

Choosing a data type is a decision about *range* (how big can the value get), *sign* (can it be negative), *precision* (can it be fractional) and *meaning* (is it a number to calculate with, or an identifier). Values that never change during a run are constants and belong in one place. Rules that every output must obey (invariants) become test cases later.

Procedure / Algorithm

1. Read the problem statement and list every value that enters the program (inputs).
2. List every value the program must calculate and print (outputs).
3. Separate values that never change during a run (constants).
4. For each item decide: can it be negative, fractional or very large?
5. Write the invariants a correct bill must satisfy, and state the assumptions.

Program / Source Code

Not applicable - Task 1 is an analysis task. The data identified here is turned into declarations in Tasks 2 and 3.

Analysis / Discussion

(a) Inputs and outputs

Data item	Role	Justification
Household number	Input	Identifier, never used in arithmetic; positive whole number with no fraction. (If leading zeros mattered it would be text.)
Category (e.g. domestic / commercial)	Input	A small fixed set of choices, so an enumerated value, not free text.
Previous reading	Input	Whole number of units read by the 16-bit meter register: 0 to 65535, never negative.
Current reading	Input	Same range. It can be *smaller* than the previous reading when the register has rolled over after 65535.
Units consumed	Output	(current - previous) modulo 65536; always 0 to 65535, never negative.
Energy charge	Output	Sum of slab charges; computed in paise (whole number) because rates such as Rs 2.25 are fractional in rupees.
Fixed charge	Output / constant	Rs 50 = 5000 paise, charged to every bill, even for 0 units.
Total bill	Output	Fixed charge + energy charge, in paise. Totals for a whole district reach tens of millions of rupees.

(b) Values that never change (constants)

- Slab limits: 100 units and 200 units.
- Rates: 0 paise (first slab), 225 paise (101-200), 450 paise (above 200).
- Fixed charge: 5000 paise; paise per rupee: 100; meter register modulus: 65536 (16-bit counter).
- They are constants because a tariff revision must be made in **one place only**, and no calculation may modify them.

(c) Negative, fractional or very large

- **Negative:** none of the stored values. Only the *intermediate* difference current - previous becomes negative after a rollover, and it must be wrapped back to a non-negative value.
- **Fractional:** rupee rates (Rs 2.25, Rs 4.50). Storing them as paise removes the fraction completely.
- **Very large:** the total collected for a district (thousands of bills x thousands of rupees x 100 paise) exceeds the 16-bit and can approach the 32-bit range, so a 64-bit-capable type is used for money.

(d) Rules every correct bill must satisfy

- A bill is never negative and is never less than the fixed charge (Rs 50).
- Total = fixed charge + energy charge (the total equals the sum of its parts).
- Bill never decreases when units increase; at slab boundaries (100, 200) the bill is continuous, with no jump.
- Units consumed are always between 0 and 65535; a rollover gives a small positive number, never a negative one.
- Sum of the bills of n households equals the sum of the individual totals, exactly (no rounding drift).

(e) Assumptions

- Readings are whole units (kWh) read from a 16-bit register; at most **one** rollover happens in a billing cycle.
- The slab tariff of this assignment applies to every category; category is stored for the record.
- Fixed charge applies even when consumption is 0. Invalid input (reading above 65535, negative) is rejected.
- All money is stored as an integer number of paise; rupees and paise are separated only when printing.

Alternative concept and complexity: readings stored as text ("01530")

If readings were stored as character strings, the program would have to convert each string to a number (a loop over the d digits: *value* = *value*10* + *digit*) before subtracting, and a comparison of two strings costs a character-by-character scan. **Time:** O(d) per conversion and per comparison, against O(1) for integers already in binary. **Space:** d + 1 bytes per reading (the extra byte is the terminating '\0'), e.g. 6 bytes for "01530", against 2 bytes for an unsigned short. Text keeps leading zeros, but billing needs arithmetic, so the integer representation is both faster and smaller.

	Integer (unsigned short)	**Text (d digits)**
Space per reading	2 bytes (O(1))	d + 1 bytes (O(d))
Time to convert	None	O(d)
Time to compare / subtract	O(1)	O(d) (after conversion)

Result

All inputs, outputs, constants, rules and assumptions of the billing problem were identified and justified. The analysis fixes two design decisions used in the rest of the assignment: readings are **unsigned 16-bit integers** and money is an **integer count of paise**.

TASK 2 - Choose the Data Types (4 Marks, K4)

Task / Problem Statement

Select a C data type for every data item of Task 1; show sizes with sizeof and ranges with limits.h / float.h; explain why money should not be float rupees; explain what happens to a 16-bit unsigned meter register after 65535; show one case where float gives a wrong total for 4,000 bills. Complete TODO 1 of `consumption()` without if-else.

Aim

To choose types that are large enough, exact enough and no larger than necessary, and to demonstrate an overflow/wrap-around and a precision problem with their resolution.

Concept / Theory

Integer types have a fixed number of bits. An **unsigned** type of n bits does all arithmetic **modulo 2^n**: after its maximum it wraps to 0 and a result below 0 wraps up from the top. This is well defined in C and exactly mirrors an odometer-style meter. **float** keeps only about 6-7 significant decimal digits and stores most decimals (such as 0.1 or 1234.56) only approximately in binary, so every addition can add a small error. An integer count of paise is exact.

Procedure / Algorithm

1. Declare `typedef unsigned short meter_t;` for the 16-bit register.
2. Compute `current - previous`. Both operands are promoted to int, so after a rollover the result is negative (e.g. -65280).
3. Convert the result back to meter_t: the value is reduced modulo 65536 (-65280 + 65536 = 256), giving the true units used.
4. Return the value as unsigned int. No if-else is needed.
5. Print sizeof/limits and test both cases; compare float and long totals for 4,000 bills.

Data-type table (sizes measured with sizeof on this system)

Data item	Type chosen	Size	Range (limits.h / float.h)	Sufficient?
Meter readings	unsigned short (meter_t)	2 bytes	0 to 65535	Yes - exactly the 16-bit register
Units consumed	unsigned int	4 bytes	0 to 4,294,967,295	Yes (max 65535)
Household number	unsigned int	4 bytes	0 to 4,294,967,295	Yes for any district
Category	enum / char	4 / 1 byte	small set of values	Yes
Money (bill, charges, rates, district total)	long (paise)	8 bytes	-9.22e18 to 9.22e18	Yes: 4,000 bills x Rs 10,000 = 4e9 paise would overflow a 32-bit int signed range (2.1e9), so long is used

Program / Source Code - consumption.c (TODO 1 completed)

```
#include <stdio.h>
#include <limits.h>

/* The meter register is a 16-bit counter: after 65535 it rolls over to 0. */
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current)
{
    /* current - previous is computed as int (integer promotion); converting
       the result back to the 16-bit unsigned type reduces it modulo 65536,
       so a rolled-over register gives the correct number of units. */
    return (meter_t)(current - previous);
}

int main(void)
{
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120)); /* rolled over */
    return 0;
}
```

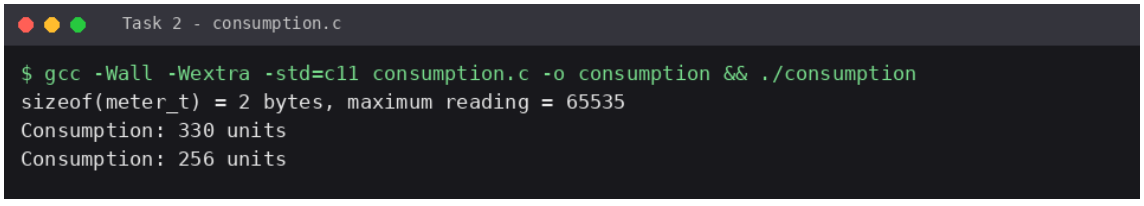
Program / Source Code - types_demo.c (sizes, ranges, float problem)

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("%-20s %-6s %s\n", "Type", "Bytes", "Range");
    printf("%-20s %-6zu %d to %d\n", "short", sizeof(short), SHRT_MIN, SHRT_MAX);
    printf("%-20s %-6zu 0 to %u\n", "unsigned short", sizeof(unsigned short), (unsigned)USHRT_MAX);
    printf("%-20s %-6zu %d to %d\n", "int", sizeof(int), INT_MIN, INT_MAX);
    printf("%-20s %-6zu 0 to %u\n", "unsigned int", sizeof(unsigned int), UINT_MAX);
    printf("%-20s %-6zu %ld to %ld\n", "long", sizeof(long), LONG_MIN, LONG_MAX);
    printf("%-20s %-6zu %lld to %lld\n", "long long", sizeof(long long), LLONG_MIN, LLONG_MAX);
    printf("%-20s %-6zu +/-%e, %d digits\n", "float", sizeof(float), FLT_MAX, FLT_DIG);
    printf("%-20s %-6zu +/-%e, %d digits\n", "double", sizeof(double), DBL_MAX, DBL_DIG);

    /* 4,000 bills of Rs 1234.56 summed as float rupees and as long paise */
    float  total_f = 0.0f;
    long   total_p = 0L;
    for (int i = 0; i < 4000; i++) {
        total_f += 1234.56f;
        total_p += 123456L;
    }
    printf("\nfloat total (4000 x Rs 1234.56) = Rs %.2f\n", total_f);
    printf("long  total (4000 x 123456 paise) = Rs %ld.%02ld\n", total_p / 100, total_p % 100);
    printf("Correct total                     = Rs 4938240.00\n");
    return 0;
}
```

Output Screenshot



Task 2 - types_demo.c

```
$ gcc -Wall -std=c11 types_demo.c -o types_demo && ./types_demo
Type                Bytes  Range
short               2      -32768 to 32767
unsigned short      2      0 to 65535
int                 4      -2147483648 to 2147483647
unsigned int        4      0 to 4294967295
long                8      -9223372036854775808 to 9223372036854775807
long long           8      -9223372036854775808 to 9223372036854775807
float               4      +/-3.402823e+38, 6 digits
double              8      +/-1.797693e+308, 15 digits

float total (4000 x Rs 1234.56) = Rs 4938053.00
long  total (4000 x 123456 paise) = Rs 4938240.00
Correct total                   = Rs 4938240.00
```

Analysis / Discussion

Why the second result is 256 and not negative. consumption(65400, 120): 120 - 65400 = -65280 as an int. Converting to the 16-bit unsigned meter_t gives -65280 + 65536 = **256**. This is correct: the meter went 65400 -> 65535 (135 units), then to 0 (1 unit), then to 120, i.e. 135 + 1 + 120 = 256 units. After 65535 an unsigned 16-bit register wraps to 0.

Why money must not be float rupees. Values like 2.25 are fine, but most decimals (e.g. 1234.56, 0.10) have no exact binary representation, and a float has only about 6 significant digits. Adding 1234.56 4,000 times gave **Rs 4,938,053.00** instead of the correct **Rs 4,938,240.00** - a loss of Rs 187 - because once the total is large, each small addition is rounded. Storing **paise in a long** (123456 per bill) gives exactly 4,938,240.00, since integer addition has no rounding.

Alternative concept: money as double rupees. The billing solution would compute rate x units in double and **round to the nearest paise once per bill** (e.g. `round(x*100)/100`) before adding to the total, and again when printing. Time per addition is O(1) in both designs. Space per value is 8 bytes for double and 8 bytes for long on this system (4 bytes if int paise were enough). The difference is accuracy: with paise the error after n bills is exactly 0, whereas with double the representation error of each bill accumulates, growing roughly in proportion to n (about the square root of n if the errors are random), so a district total can be off by several rupees.

Result

Output matches the expected output (2 bytes / 65535; 330 units; 256 units). The program completes TODO 1 using only unsigned 16-bit arithmetic, and the float problem was shown and resolved with long paise.

TASK 3 - Define the Constants (3 Marks, K3)

Task / Problem Statement

Define tariff values and limits as constants so a tariff revision is made in one place only. Complete TODO 1 of `compute_bill()` using only the macros defined at the top, run it, and check that changing RATE2_PAISE to 230 changes every affected bill consistently.

Aim

To remove magic numbers from the billing code by using well-named constants and to verify that a single edit changes the whole program consistently.

Concept / Theory

	#define	const variable	enum
Kind	Text substitution by the preprocessor	A real object with a type	Named integer constants
Type checking	None (just text)	Yes	int only
Scope	From definition to end of file; ignores blocks	Normal C scope (block / file)	Normal C scope
Debugging	Name not visible to the debugger (replaced earlier)	Visible; can be inspected	Visible
Memory	None	May occupy storage	None
Best used for	Simple tariff values, array sizes	Typed values, values needing an address	Groups of related codes (e.g. categories)

Naming convention: constants in UPPER_CASE with underscores, with the unit in the name (RATE2_PAISE, FIXED_CHARGE_PAISE, SLAB1_UNITS) so a wrong-unit mistake is visible. A suffix such as `5000L` makes the literal a long.

Procedure / Algorithm

1. Define SLAB1_UNITS (100), SLAB2_UNITS (200), RATE1/2/3_PAISE (0, 225, 450) and FIXED_CHARGE_PAISE (5000L) at the top.
2. Start paise with FIXED_CHARGE_PAISE.
3. If units exceed SLAB2_UNITS, add (units - SLAB2_UNITS) x RATE3_PAISE and cap units at SLAB2_UNITS.
4. If units exceed SLAB1_UNITS, add (units - SLAB1_UNITS) x RATE2_PAISE and cap units at SLAB1_UNITS.
5. Add the remaining units x RATE1_PAISE and return paise; print as Rs paise/100 . paise%100 (two digits).

Program / Source Code - billing.c (TODO 1 completed)

```
#include <stdio.h>

#define SLAB1_UNITS 100        /* first 100 units */
#define SLAB2_UNITS 200        /* units 101 to 200 */
#define RATE1_PAISE 0          /* first slab is free */
#define RATE2_PAISE 225 /* Rs 2.25 per unit */
#define RATE3_PAISE 450        /* Rs 4.50 per unit above 200 */
#define FIXED_CHARGE_PAISE 5000L /* Rs 50 fixed charge */

/* Returns the bill in paise, so that no rounding error can occur. */
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {                      /* units above slab 2 */
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
        units = SLAB2_UNITS;
    }
    if (units > SLAB1_UNITS) {                      /* units 101..200 */
        paise += (long)(units - SLAB1_UNITS) * RATE2_PAISE;
        units = SLAB1_UNITS;
    }
    paise += (long)units * RATE1_PAISE;             /* the rest, first slab */

    return paise;
}

int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n", tests[i], p / 100, p % 100);
    }
    return 0;
}
```

Output Screenshot (RATE2_PAISE = 225, then 230)

```
Task 3 - billing.c

$ gcc -Wall -std=c11 billing.c -o billing && ./billing
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 162.50
200 units -> Rs 275.00
350 units -> Rs 950.00
$ gcc -Wall -std=c11 billing_230.c -o billing230 && ./billing230   # RATE2_PAISE = 230
  0 units -> Rs 50.00
100 units -> Rs 50.00
150 units -> Rs 165.00
200 units -> Rs 280.00
350 units -> Rs 955.00
```

Analysis / Discussion

Hand check (RATE2 = 225): 150 units = 5000 + 50 x 225 = 16250 paise = Rs 162.50. 350 units = 5000 + 150 x 450 + 100 x 225 = 95000 paise = Rs 950.00. With RATE2_PAISE = 230 only the bills that use slab 2 change: 150 units = 5000 + 50 x 230 = Rs 165.00 (+Rs 2.50), 200 units = Rs 280.00 (+Rs 5.00), 350 units = Rs 955.00 (+Rs 5.00); the bills for 0 and 100 units stay Rs 50.00. Only **one line** was edited, so the revision is consistent.

Alternative concept: const variables or enum. With `static const long RATE2_PAISE = 225;` the compiler type-checks every use, the value is visible in the debugger, and it has a normal scope; it may use a few bytes of memory (a #define uses none). An enum would only hold int-sized values, so it suits the category codes but not a long fixed charge in general. Running time is not affected: an optimising compiler

replaces a const with its value, so each bill is still computed in O(1) time with a constant number of operations.

Result

Output matches the expected output (Rs 50.00, 50.00, 162.50, 275.00, 950.00). The tariff is a single, well-named set of #define constants, and the RATE2_PAISE = 230 test shows that every affected bill changes consistently.

TASK 4 - Use Macros Safely (4 Marks, K4)

Task / Problem Statement

Write macros for small repeated calculations (rupees to paise, larger of two values), show how the preprocessor expands them with `gcc -E`, and explain the SQUARE(a+1) and MAX(i++, j) pitfalls and when a function is better.

Aim

To write correct, fully parenthesised macros and to understand, with evidence, why careless macros give wrong results.

Concept / Theory

A macro is **text substitution** done by the preprocessor before compilation: the argument text is pasted in, not evaluated first. So operator precedence of the surrounding expression can change the meaning, and an argument that appears twice in the macro body is evaluated twice. Parentheses around every parameter and around the whole body restore the intended order; a function evaluates each argument exactly once and is type-checked.

Procedure / Algorithm

1. Write RUPEES_TO_PAISE(r) as ((long)(r) * 100L), MAX(a,b) and SQUARE(x) with every parameter and the whole body in parentheses.
2. Also write the faulty SQUARE_BAD(x) = x * x to demonstrate the problem.
3. Call SQUARE_BAD(a+1) and SQUARE(a+1) with a = 3, and MAX(i++, j) with i = 5, j = 3.
4. Run `gcc -E` to see the expanded text.
5. Compare with static inline function versions, which evaluate each argument once.

Program / Source Code - macros.c

```
#include <stdio.h>

#define RUPEES_TO_PAISE(r)  ((long)(r) * 100L)
#define SQUARE_BAD(x)       x * x
#define SQUARE(x)           ((x) * (x))
#define MAX(a, b)           (((a) > (b)) ? (a) : (b))

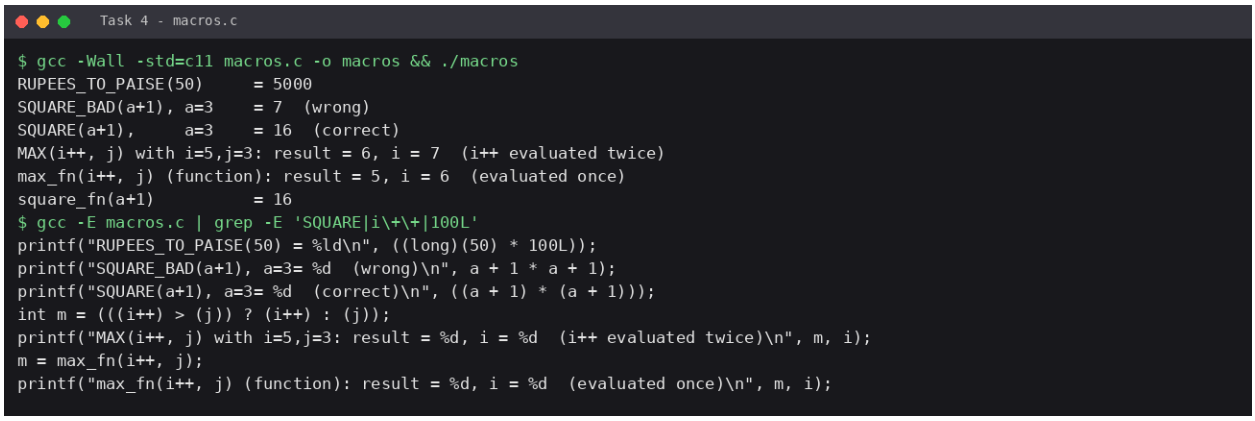
static inline int  max_fn(int a, int b)     { return a > b ? a : b; }
static inline long square_fn(long x)        { return x * x; }

int main(void)
{
    int a = 3;
    printf("RUPEES_TO_PAISE(50)     = %ld\n", RUPEES_TO_PAISE(50));
    printf("SQUARE_BAD(a+1), a=3    = %d  (wrong)\n", SQUARE_BAD(a + 1));
    printf("SQUARE(a+1),     a=3    = %d  (correct)\n", SQUARE(a + 1));

    int i = 5, j = 3;
    int m = MAX(i++, j);
    printf("MAX(i++, j) with i=5,j=3: result = %d, i = %d  (i++ evaluated twice)\n", m, i);

    i = 5;
    m = max_fn(i++, j);
    printf("max_fn(i++, j) (function): result = %d, i = %d  (evaluated once)\n", m, i);
    printf("square_fn(a+1)          = %ld\n", square_fn(a + 1));
    return 0;
}
```

Output Screenshot (program output and gcc -E expansion)



Analysis / Discussion

- **Preprocessor expansion (gcc -E):** RUPEES_TO_PAISE(50) becomes `((long)(50) * 100L)`, and MAX(i++, j) becomes the text shown above, with i++ written twice.
- **SQUARE(x) as x*x:** SQUARE_BAD(a+1) expands to `a + 1 * a + 1`. Multiplication binds tighter, so with a = 3 it is 3 + 3 + 1 = **7**, not 16. With parentheses it becomes `((a + 1) * (a + 1))` = **16**.
- **MAX(i++, j):** the macro body uses a twice, so i++ is evaluated twice: i = 5 is compared, incremented to 6, then i++ is evaluated again and the macro returns 6, leaving i = 7 (one increment was intended). The function max_fn(i++, j) returns 5 and leaves i = 6.
- **When a function is better:** when arguments may have side effects (i++, function calls), when type checking is wanted, when the body is long, when the debugger should step into it, or when the address of the code is needed. Macros suit tiny constant-like expressions.

Alternative concept: static inline functions. MAX and SQUARE become `static inline int max_fn(int a, int b)` and `static inline long square_fn(long x)`. Arguments are evaluated once and types are checked. **Time:** a normal call costs a few extra instructions (push arguments, jump, return) but is still O(1); an expanded macro has no call cost, and a compiler that inlines the function gives the same speed. **Space:** a macro body of size s used k times is copied k times, giving code size O(k x s); a non-inlined function keeps one copy plus k small call instructions, i.e. O(s + k).

Result

The correct, fully parenthesised macros give SQUARE(a+1) = 16 and RUPEES_TO_PAISE(50) = 5000, while the gcc -E output explains why the unparenthesised version gives 7 and why MAX(i++, j) increments i twice. I learned to parenthesise every parameter and to avoid passing expressions with side effects to a macro.

Test Cases / Validation Table

Record different inputs, expected results, actual results, and whether each test passed.

Test No.	Input / Scenario	Expected Output	Actual Output	Status
1	consumption(1200, 1530) - normal reading	330 units	330 units	Pass
2	consumption(65400, 120) - register rolled over	256 units	256 units	Pass
3	compute_bill(0) - no consumption	Rs 50.00 (fixed charge only)	Rs 50.00	Pass
4	compute_bill(100) - end of free slab	Rs 50.00	Rs 50.00	Pass
5	compute_bill(150) - inside slab 2	Rs 162.50	Rs 162.50	Pass
6	compute_bill(200) - end of slab 2	Rs 275.00	Rs 275.00	Pass
7	compute_bill(350) - into slab 3	Rs 950.00	Rs 950.00	Pass
8	RATE2_PAISE = 230, compute_bill(150)	Rs 165.00	Rs 165.00	Pass
9	SQUARE(a+1), a = 3	16	16	Pass
10	4,000 x Rs 1234.56 as long paise	Rs 4938240.00	Rs 4938240.00	Pass

Final Submission Checklist

- Student and course details completed
- Task statement, aim, theory, and algorithm included
- Program code is readable and runs successfully
- Output screenshot included
- Analysis and result written in own words
- Test cases completed where applicable
- GitHub repository link added